

OPEN TECH NOTES

loạt tài liệu kỹ thuật miễn phí cho cộng đồng

FINE-TUNING

Huấn luyện tiếp mô hình cho bài toán riêng của bạn

Phương pháp, quy trình thực tế, và khi nào nên hay không nên fine-tune

Phần 4 trong loạt bài Kỹ thuật AI Engineering

Biên soạn cho cộng đồng IT Việt Nam · Tháng 8, 2026

Mục lục

Mở đầu	3
1. Fine-tuning là gì và khi nào cần đến nó	3
2. Các phương pháp fine-tuning phổ biến	4
2.1. Full fine-tuning.....	4
2.2. PEFT và LoRA.....	4
3. Quy trình fine-tuning trong thực tế	5
3.1. Chuẩn bị dữ liệu huấn luyện	5
3.2. Huấn luyện & siêu tham số	6
3.3. Đánh giá trước khi triển khai	6
4. Fine-tuning so với prompting và RAG.....	6
5. Rủi ro & giới hạn thực tế	6
5.1. Catastrophic forgetting.....	6
5.2. Chi phí, bảo trì, và drift theo thời gian.....	7
6. Checklist nhanh cho đội ngũ	7
Đọc thêm	7
Tài liệu tham khảo	8

Mở đầu

Ba phần trước của loạt bài đã đi qua ba cách phổ biến để một foundation model làm đúng việc bạn cần: chọn đúng mô hình nền (phần 1), viết prompt tốt (phần 2), và cấp thêm kiến thức bên ngoài qua RAG (phần 3). Cả ba cách đó đều có một điểm chung không thay đổi trọng số mô hình. Fine-tuning là hướng còn lại: huấn luyện tiếp một mô hình đã có sẵn trên dữ liệu của riêng bạn, để bản thân trọng số mô hình thay đổi theo hướng bạn mong muốn.

Đây thường là công cụ được cân nhắc sau cùng, không phải vì nó kém hiệu quả, mà vì chi phí và độ phức tạp vận hành cao hơn hẳn so với prompting hay RAG cần dữ liệu huấn luyện chất lượng, tài nguyên tính toán, quy trình đánh giá nghiêm túc, và kế hoạch bảo trì lâu dài. Tài liệu này trình bày khi nào fine-tuning thực sự đáng làm, các phương pháp phổ biến nhất hiện nay (đặc biệt là các kỹ thuật tiết kiệm tài nguyên như LoRA), một quy trình thực tế cho đội kỹ sư, và những rủi ro dễ bị bỏ qua. Nội dung là bản tổng hợp gốc dựa trên kiến thức chung của ngành và các nguồn công khai, có trích dẫn ở cuối bài.

1. Fine-tuning là gì và khi nào cần đến nó

Fine-tuning là quá trình huấn luyện tiếp một mô hình đã được huấn luyện sẵn (pretrained/instruct model) trên một tập dữ liệu nhỏ hơn, tập trung vào một tác vụ hoặc phong cách cụ thể khác với việc huấn luyện một mô hình từ đầu (from scratch), vốn đòi hỏi dữ liệu và tài nguyên tính toán lớn hơn nhiều bậc. Vì mô hình đã có sẵn nền tảng ngôn ngữ và kiến thức chung từ giai đoạn pretraining, fine-tuning chỉ cần điều chỉnh một phần nhỏ hành vi của nó.

Khung quyết định giới thiệu ở phần 2 của loạt bài (prompting → RAG → fine-tuning) vẫn là điểm khởi đầu hợp lý, nhưng đáng nói rõ ba tình huống mà fine-tuning thường là lựa chọn đúng đắn, không chỉ là "phương án cuối cùng":

- Hành vi/định dạng cần nhất quán tuyệt đối ở quy mô lớn: khi một chỉ dẫn trong prompt vẫn bị bỏ sót ở một tỷ lệ nhỏ dù đã diễn đạt rất kỹ, và tỷ lệ lỗi đó không chấp nhận được ở quy mô hàng triệu lượt gọi.
- Cần giảm độ dài prompt và độ trễ: khi một hành vi phải lặp lại chỉ dẫn dài dòng trong mọi prompt, fine-tuning có thể "khắc" hành vi đó vào trọng số, giúp prompt production ngắn hơn, rẻ hơn, và nhanh hơn.
- Cần một phong cách/giọng điệu đặc thù rất khó mô tả bằng lời: ví dụ mô phỏng văn phong của một thương hiệu cụ thể qua hàng trăm ví dụ có sẵn, để đạt được bằng cách "cho xem ví dụ để học theo" hơn là cố mô tả bằng văn bản.

Ngược lại, nếu vấn đề là thiếu kiến thức cụ thể, hay thay đổi theo thời gian, RAG gần như luôn là lựa chọn phù hợp hơn fine-tuning không phải là cách hiệu quả để "nhồi" kiến thức thực tế vào mô hình, và dễ dẫn đến ảo giác nếu dùng sai mục đích này.

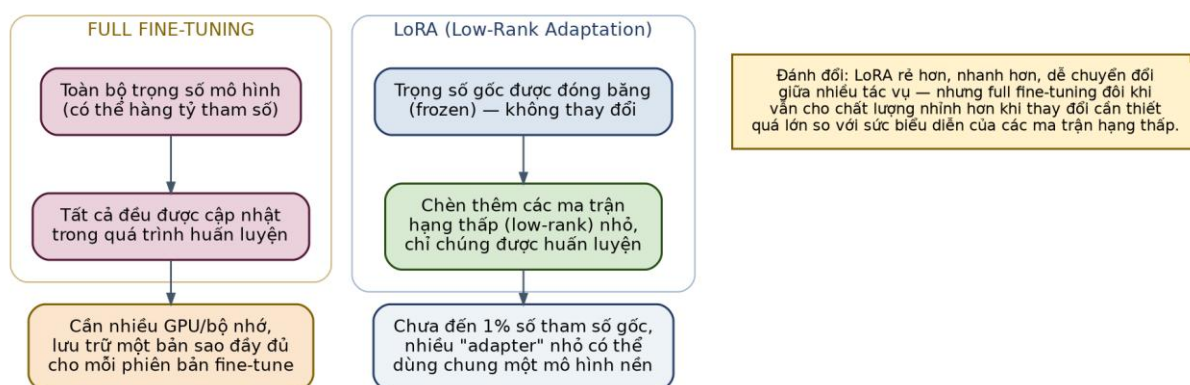
2. Các phương pháp fine-tuning phổ biến

2.1. Full fine-tuning

Full fine-tuning cập nhật toàn bộ trọng số của mô hình trong quá trình huấn luyện tiếp về nguyên tắc cho khả năng biểu diễn (khả năng học được thay đổi phức tạp) cao nhất, nhưng đòi hỏi tài nguyên tương đương với việc huấn luyện một mô hình có cùng kích thước: nhiều GPU, nhiều bộ nhớ, và mỗi phiên bản fine-tune tạo ra một bản sao đầy đủ của mô hình cần lưu trữ và phục vụ riêng. Với các mô hình từ vài tỷ tham số trở lên, chi phí này thường vượt quá khả năng của phần lớn đội kỹ sư ứng dụng, trừ khi có hạ tầng và ngân sách chuyên biệt.

2.2. PEFT và LoRA

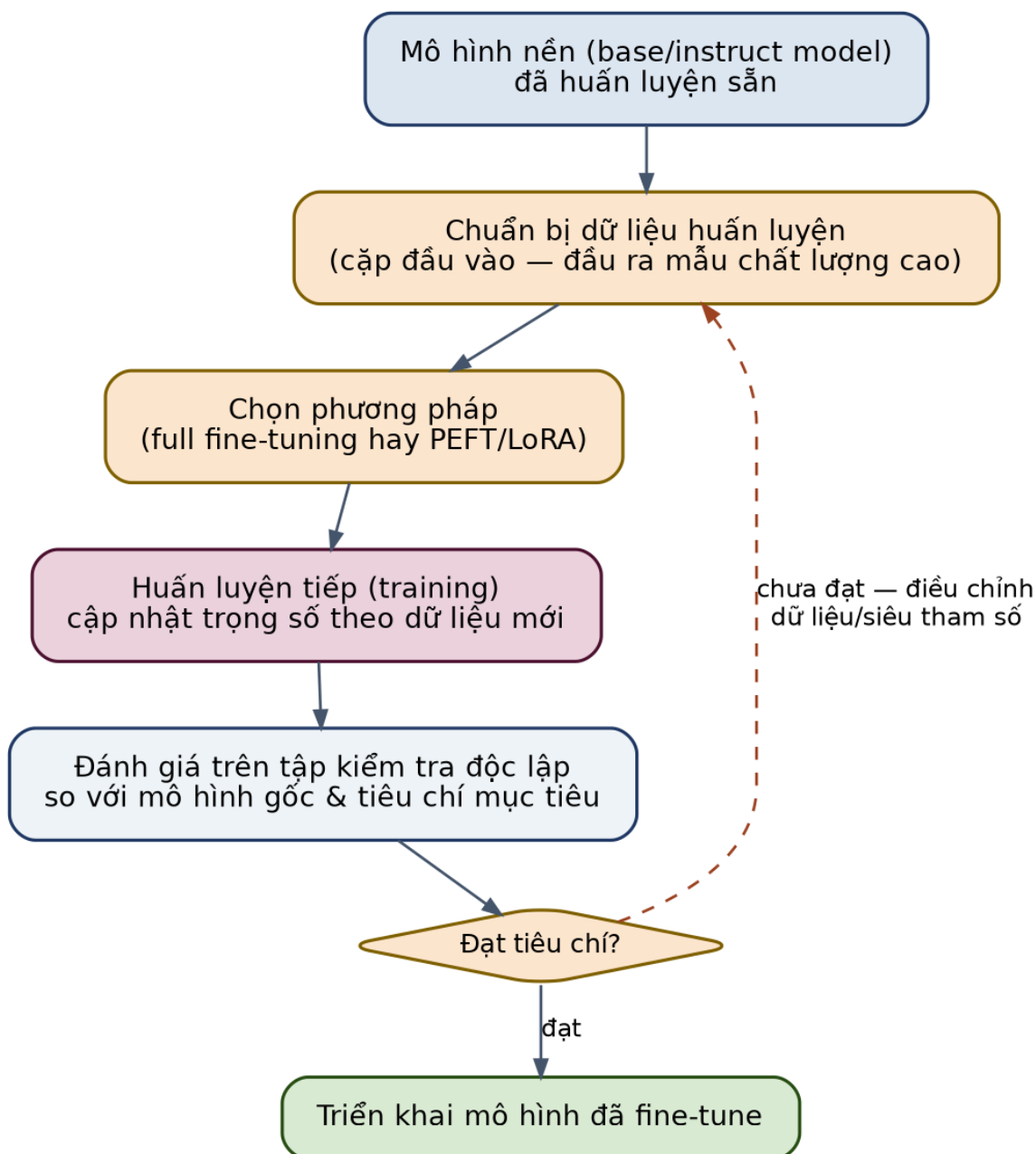
Parameter-Efficient Fine-Tuning (PEFT) là nhóm kỹ thuật chỉ huấn luyện một phần rất nhỏ tham số, trong khi giữ nguyên (đóng băng) phần lớn trọng số gốc giảm mạnh chi phí tính toán và bộ nhớ mà vẫn đạt chất lượng gần với full fine-tuning trong nhiều trường hợp thực tế. Kỹ thuật phổ biến nhất trong nhóm này là LoRA (Low-Rank Adaptation, Hu và cộng sự, 2021): thay vì cập nhật trực tiếp ma trận trọng số gốc, LoRA chèn thêm các cặp ma trận có hạng thấp (low-rank) nhỏ vào một số lớp của mô hình, và chỉ huấn luyện các ma trận nhỏ đó.



Hình 1. So sánh full fine-tuning (cập nhật toàn bộ trọng số) với LoRA (chỉ huấn luyện các ma trận hạng thấp được chèn thêm, trọng số gốc đóng băng).

Vì số tham số cần huấn luyện của LoRA thường chưa đến 1% so với mô hình gốc, chi phí tính toán và dung lượng lưu trữ giảm mạnh mỗi "adapter" LoRA cho một tác vụ chỉ nặng vài chục đến vài trăm MB, có thể hoán đổi linh hoạt trên cùng một mô hình nền dùng chung, thay vì phải lưu nhiều bản sao đầy đủ. Một biến thể đáng chú ý là QLoRA (Dettmers và cộng sự, 2023), kết hợp LoRA với việc lượng tử hoá (quantization) trọng số gốc xuống độ chính xác thấp hơn, giúp fine-tune được cả những mô hình lớn trên phần cứng phổ thông hơn.

3. Quy trình fine-tuning trong thực tế



Hình 2. Quy trình fine-tuning điển hình: chuẩn bị dữ liệu, chọn phương pháp, huấn luyện, đánh giá, và lặp lại nếu chưa đạt tiêu chí.

3.1. Chuẩn bị dữ liệu huấn luyện

Chất lượng dữ liệu huấn luyện quyết định kết quả nhiều hơn bất kỳ lựa chọn kỹ thuật nào khác một tập dữ liệu nhỏ nhưng sạch, đúng định dạng, và đại diện tốt cho phân bố thực tế thường cho kết quả tốt hơn một tập lớn nhưng lẫn tạp. Dữ liệu thường ở dạng cặp đầu vào — đầu ra mẫu (ví dụ câu hỏi và câu trả lời lý tưởng), cần được rà soát thủ công một phần để phát hiện lỗi nhân, trùng lặp, hoặc thiên lệch (bias) không mong muốn có thể bị mô hình học theo. Cần tách riêng một phần dữ liệu không đưa vào huấn luyện, dành cho việc đánh giá độc lập ở bước 3.3.

3.2. Huấn luyện & siêu tham số

Các siêu tham số (hyperparameter) quan trọng cần điều chỉnh gồm tốc độ học (learning rate thường thấp hơn nhiều so với pretraining), số epoch (số lượt duyệt qua toàn bộ dữ liệu quá nhiều dễ dẫn đến overfitting trên tập huấn luyện nhỏ), và với LoRA, hạng (rank) của các ma trận được chèn thêm hạng cao hơn tăng khả năng biểu diễn nhưng cũng tăng chi phí. Phần lớn nhà cung cấp mô hình lớn hiện cung cấp dịch vụ fine-tuning được quản lý (managed fine-tuning API), giúp đội kỹ sư ứng dụng không cần tự vận hành hạ tầng huấn luyện, chỉ cần cung cấp dữ liệu và một số siêu tham số cơ bản.

3.3. Đánh giá trước khi triển khai

So sánh mô hình đã fine-tune với mô hình gốc trên cùng một bộ đánh giá độc lập (không có trong dữ liệu huấn luyện) là bước bắt buộc trước khi triển khai không chỉ đo tác vụ mục tiêu có cải thiện hay không, mà còn cần đo cả những khả năng chung khác của mô hình có bị suy giảm hay không (xem catastrophic forgetting ở phần 5.1). Nội dung đánh giá mô hình một cách hệ thống sẽ được trình bày đầy đủ trong tài liệu riêng của loạt bài này.

4. Fine-tuning so với prompting và RAG

Ba công cụ không loại trừ nhau hệ thống production thực tế thường kết hợp cả ba: một mô hình đã fine-tune để nắm vững định dạng/phong cách đầu ra mong muốn, vẫn nhận ngữ cảnh qua RAG để có kiến thức cập nhật, và vẫn dùng prompt engineering tốt cho từng lượt gọi cụ thể. Bảng dưới đây tóm tắt điểm khác biệt cốt lõi khi cân nhắc dùng công cụ nào cho một vấn đề cụ thể: prompting thay đổi tức thời và rẻ nhất nhưng giới hạn ở phạm vi một prompt; RAG cập kiến thức bên ngoài cập nhật được nhưng không thay đổi hành vi nền tảng của mô hình; fine-tuning thay đổi chính hành vi/phong cách của mô hình một cách bền vững nhưng tốn chi phí huấn luyện, dữ liệu, và cần bảo trì khi mô hình nền được nâng cấp.

Một sai lầm thường gặp là dùng fine-tuning để "vá" một vấn đề đáng lẽ prompting đã giải quyết được trước khi đầu tư vào fine-tuning, nên chắc chắn đã thử prompt engineering kỹ lưỡng (xem phần 2–3 của tài liệu Prompt Engineering) và, nếu vấn đề liên quan đến kiến thức cụ thể, đã thử RAG. Ngược lại, một sai lầm khác là cố kéo dài prompt engineering hoặc RAG cho một vấn đề thực chất là hành vi/phong cách cần nhất quán tuyệt đối ở quy mô lớn trường hợp đó, đầu tư sớm vào fine-tuning thường tiết kiệm hơn về lâu dài.

5. Rủi ro & giới hạn thực tế

5.1. Catastrophic forgetting

Catastrophic forgetting ("quên thảm khốc") là hiện tượng mô hình, khi được huấn luyện tiếp quá mạnh trên một tập dữ liệu hẹp, dần đánh mất những khả năng chung đã học được từ giai đoạn pretraining. Hiện tượng này đã được ghi nhận trong nghiên cứu về mạng nơ-ron từ trước cả thời đại LLM (Kirkpatrick và cộng sự, 2017), và vẫn là rủi ro thực tế với fine-tuning foundation model ngày nay. Biểu hiện thường gặp: mô hình fine-tune giỏi lên rõ rệt ở tác vụ mục tiêu, nhưng lại kém đi ở những tác vụ khác vốn không liên quan trực tiếp đến dữ liệu huấn luyện. Điều này chỉ phát hiện được nếu bộ đánh giá bao gồm cả các tác vụ ngoài phạm vi mục tiêu, không chỉ đo riêng tác vụ đang tối ưu.

5.2. Chi phí, bảo trì, và drift theo thời gian

Fine-tuning không phải một khoản đầu tư một lần. Khi nhà cung cấp phát hành phiên bản mô hình nền mới (thường nhanh hơn nhiều so với chu kỳ dữ liệu huấn luyện của bạn), dữ liệu fine-tuning thường cần được huấn luyện lại trên mô hình nền mới để tận dụng cải tiến. Nếu không, sản phẩm dần bị "khóa" vào một phiên bản mô hình cũ hơn, có thể sớm bị ngừng hỗ trợ. Ngoài ra, hành vi thực tế của người dùng và dữ liệu có xu hướng thay đổi theo thời gian (data drift). Một mô hình fine-tune tốt tại thời điểm huấn luyện có thể dần kém phù hợp nếu không được đánh giá lại định kỳ trên dữ liệu mới. Vì vậy, quyết định fine-tune cần đi kèm kế hoạch bảo trì, không chỉ kế hoạch huấn luyện ban đầu.

6. Checklist nhanh cho đội ngũ

- Đã thử kỹ prompt engineering và, nếu liên quan đến kiến thức cụ thể, đã thử RAG trước khi quyết định fine-tune chưa?
- Dữ liệu huấn luyện đã được rà soát chất lượng, khử trùng lặp, và tách riêng tập đánh giá độc lập chưa?
- Đã cân nhắc LoRA/PEFT trước khi đầu tư vào full fine-tuning, nếu tài nguyên là một ràng buộc?
- Bộ đánh giá trước khi triển khai có đo cả khả năng chung của mô hình, không chỉ riêng tác vụ mục tiêu (để phát hiện catastrophic forgetting)?
- Có kế hoạch huấn luyện lại khi mô hình nền được nâng cấp phiên bản mới?
- Có kế hoạch theo dõi và đánh giá lại định kỳ để phát hiện data drift sau khi triển khai?

Đọc thêm

Tài liệu này tập trung vào các nguyên tắc và quy trình nền tảng của fine-tuning. Nếu bạn muốn đào sâu hơn với một góc nhìn hệ thống, đầy đủ ví dụ mã nguồn và case study thực chiến về huấn luyện và triển khai mô hình tùy chỉnh trong bối cảnh ứng dụng production trên nền foundation model, cuốn sách sau là một nguồn tham khảo chuyên sâu đáng đọc:

"AI Engineering: Building Applications with Foundation Models" Chip Huyen, O'Reilly Media, 2025.

Bản tiếng Việt phát hành bởi Times, có bán tại các nhà sách và sàn thương mại điện tử.

Tài liệu bạn đang đọc là nội dung gốc, biên soạn độc lập dựa trên kiến thức chung của ngành và các nguồn công khai liệt kê bên dưới không phải bản dịch hay tóm tắt của cuốn sách trên.

Tài liệu tham khảo

- Hu, E. et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR, 2022 (arXiv:2106.09685).
- Dettmers, T. et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS, 2023 (arXiv:2305.14314).
- Houlisby, N. et al. "Parameter-Efficient Transfer Learning for NLP." ICML, 2019.
- Kirkpatrick, J. et al. "Overcoming Catastrophic Forgetting in Neural Networks." PNAS, 2017.
- Hugging Face. "PEFT: Parameter-Efficient Fine-Tuning." huggingface.co/docs/peft, 2026.
- OpenAI. "Fine-tuning guide." platform.openai.com/docs, 2026.